

Cybersecurity (F26)

25th March 2026

Sani Abdullahi, Ph.D.

[\[saa@mmmi.sdu.dk\]](mailto:saa@mmmi.sdu.dk)

Course Plan

Lecture	Date	Teacher	Topic
01	2026-02-04	Sani	Introduction & Lab Set-up
02	2026-02-11	Sani	Vulnerability Assessment
03	2026-02-18	Sani	Penetration Testing
04	2026-02-25	Hiraku	Firewalls and IDS
05	2026-03-04	Hiraku	Malware & SQL
06	2026-03-11	Hiraku	Denial of Service
07	2026-03-18	Sani	Social Engineering, Phishing
08	2026-03-25	Sani	Threat Modelling I
09	2026-04-08	Hiraku	Cryptography
10	2026-04-15	Hiraku	Multi-party Computation
11	2026-04-22	Jukka	EU Cybersecurity Law
12	2026-04-29	Sani	Threat Modelling II, Revision, & Exam Prep.

Information

→ Week 17 Lecture Update:

→ The lecture “Privacy-preserving Data Analysis” by Hiraku has been **replaced with** “EU Cybersecurity Law” by Jukka

→ **Remember to complete all group registrations before deadline**

→ **Remember to complete Mid-term Evaluation:** <https://pollev.com/sani>

→ Active until April 5th

→ **Common Cybersecurity Misconceptions:** Breaking into systems, writing exploits, being like hackers in movies

→ **Reality:** Security is a structured process (defense, not offence), multiple roles beyond hacking, soft skills matter too.

Cybersecurity Incident of the Week

POLITICS

Russian hackers targeting US officials and journalists on Signal – have accessed ‘thousands’ of accounts, FBI warns

Published March 21, 2026, 12:46 a.m. ET



Threat modeling insight:

- **Broken assumption:** “End-to-end encryption = secure system”
- **Missed threat:** Social engineering + identity takeover
- **Model update:** Include user interaction abuse and authentication flows as attack surfaces

Cyberattack forces Belgian hospital to transfer critical care patients

January 14th, 2026



Threat modeling insight:

- **Broken assumption:** “IT failures don’t directly affect safety”
- **Reality:** Cyber → physical impact (cyber-physical systems)
- **Model update:** Include safety-critical dependencies and operational disruption scenarios

Threat Modeling I

Learning Objectives

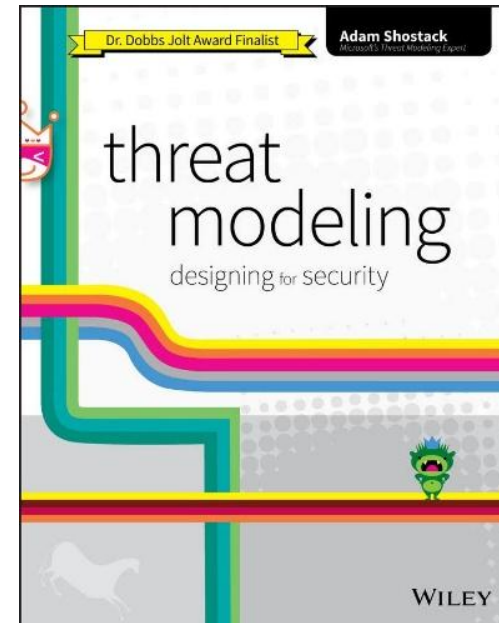
By the end of this lecture, you should be able to:

→ Understand

Threat Modeling

Common ways to find security issues:

- Wouldn't it be better to find security issues before you write a single line of code?
- But how to identify security issues?
 - Static code analysis
 - Fuzzing or other dynamic testing
 - Penetration testing / red team
 - Attack-defence training: red team/blue team
 - Wait for bug reports after release



Threat Modeling

Common ways to find security issues II:

Doing Threat Modeling

- Think about security issues early on.
- Understand your requirements better
- Avoid coding security issues
- Monitor your system / deployment

You're already threat modeling

- Whether you develop or operate
- Maybe just implicitly?
- **Key question:** how can we ensure more secure software/systems?

Theat Modeling

Aims of threat modeling

- A tool to improve security
- By addressing predictable threats ...
- Success depends on how thorough we are
 - Structure and efficiency are of essence (focusing on the big picture, not getting lost in details)
 - Make this a process!

Common threat modelling approaches

- Models that **focus on the attackers**
- Models that **focus on the assets**
- Models that **focus on the software/system**

Threat Modeling Approach

Focusing on attacks / Thinking like an attacker

What do they know? What will they do?

What are the problems associated with the “**Think like an attacker**” mentality?

- **Risky** – if you get these wrong, your threat modeling will go astray
- And you know very much about the system anyhow
- So don't start from attackers (maybe)!
- Still, you should keep them in mind for different scenarios

Threat Modeling Approach

Focusing on the assets

What are the problems associated with starting from assets as an approach to threat modeling?

- **Assets:** Valuable things – the business cares about
- **Risk of perspective:** Do you really know the asset?
- Something an attacker wants?
- Something you want to protect?
- A stepping stone?



Overlapping definitions of assets

Threat Modeling Approach

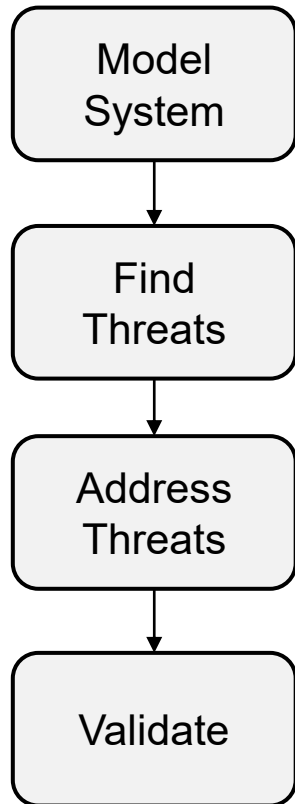
Focusing on the assets

Can be complicated

- It's hard to judge what an attacker will want:
 - Maybe they want a random computer to serve pirated music. What do you protect?
- Depends also on the attacker (different scenarios)
- Everything inside the perimeter, because every computer is stepping stone to the inner-sanctum.
So what do you learn by making an asset list? Setting up your defences.

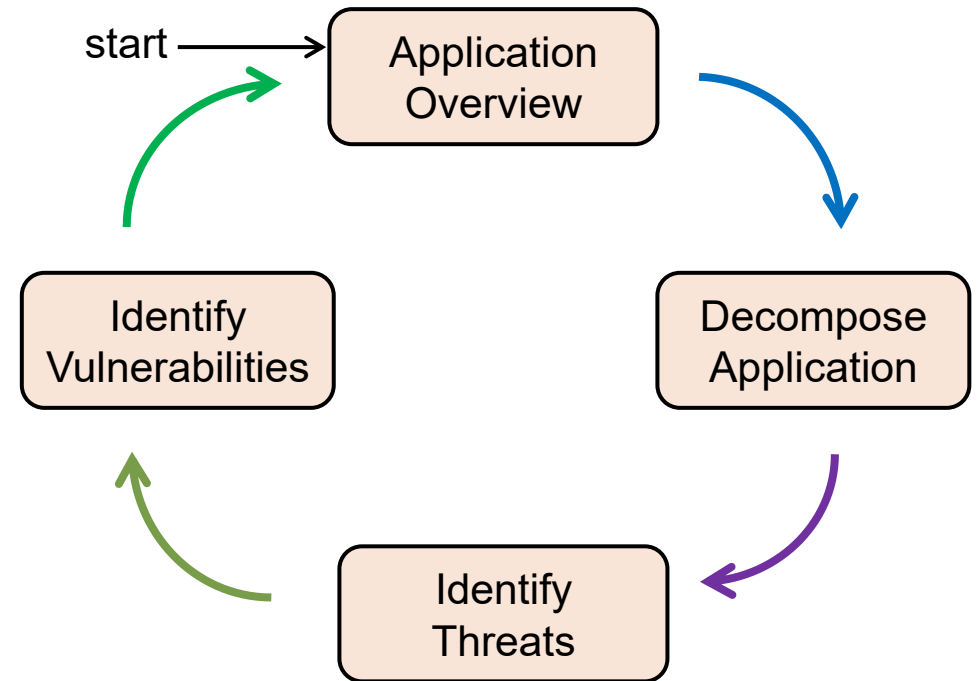
Threat Modeling

Engineering real technology



Needs an engineering approach

- Predictable
- Reliable
- Scalable (to a large product)
- Practical!



OWASP Threat Modeling

Threat modeling

What are you building/developing

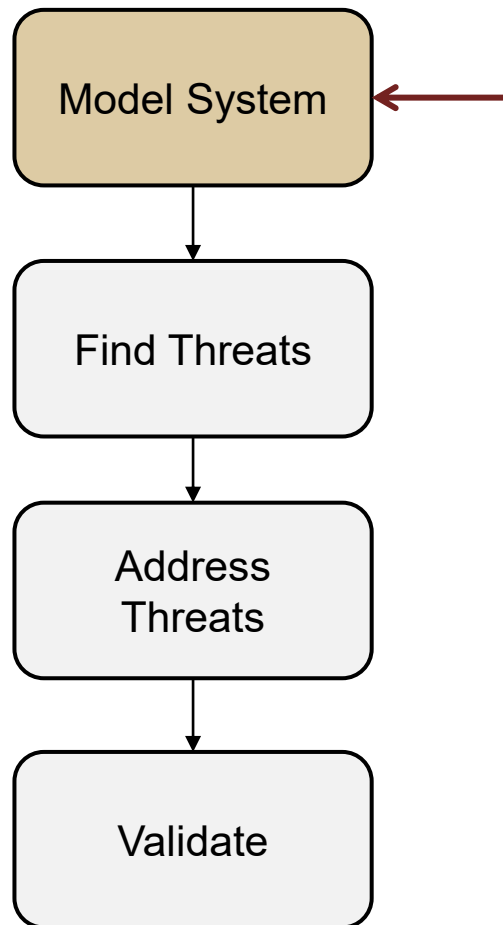
- Create a model of software/system/technology
- A model abstract away the detail, so that you can look at the whole
- Diagramming is a key approach
- Mathematical models are rare in commercial environments

Examples for modeling methods (structured "formal" diagrams)

- Data flow diagrams
- Swim lanes
- State machines
- Other UML diagrams

Models of Software/System (Threat Modeling Flow)

Threat Modeling Flow



The four-step framework

Rewriting our system to enable analysis

Threat Modeling Flow

Data Flow Diagram (DFD)

Developed in the early 70s, and still useful

- Simple: easy to learn, sketch
- Threats often follow data

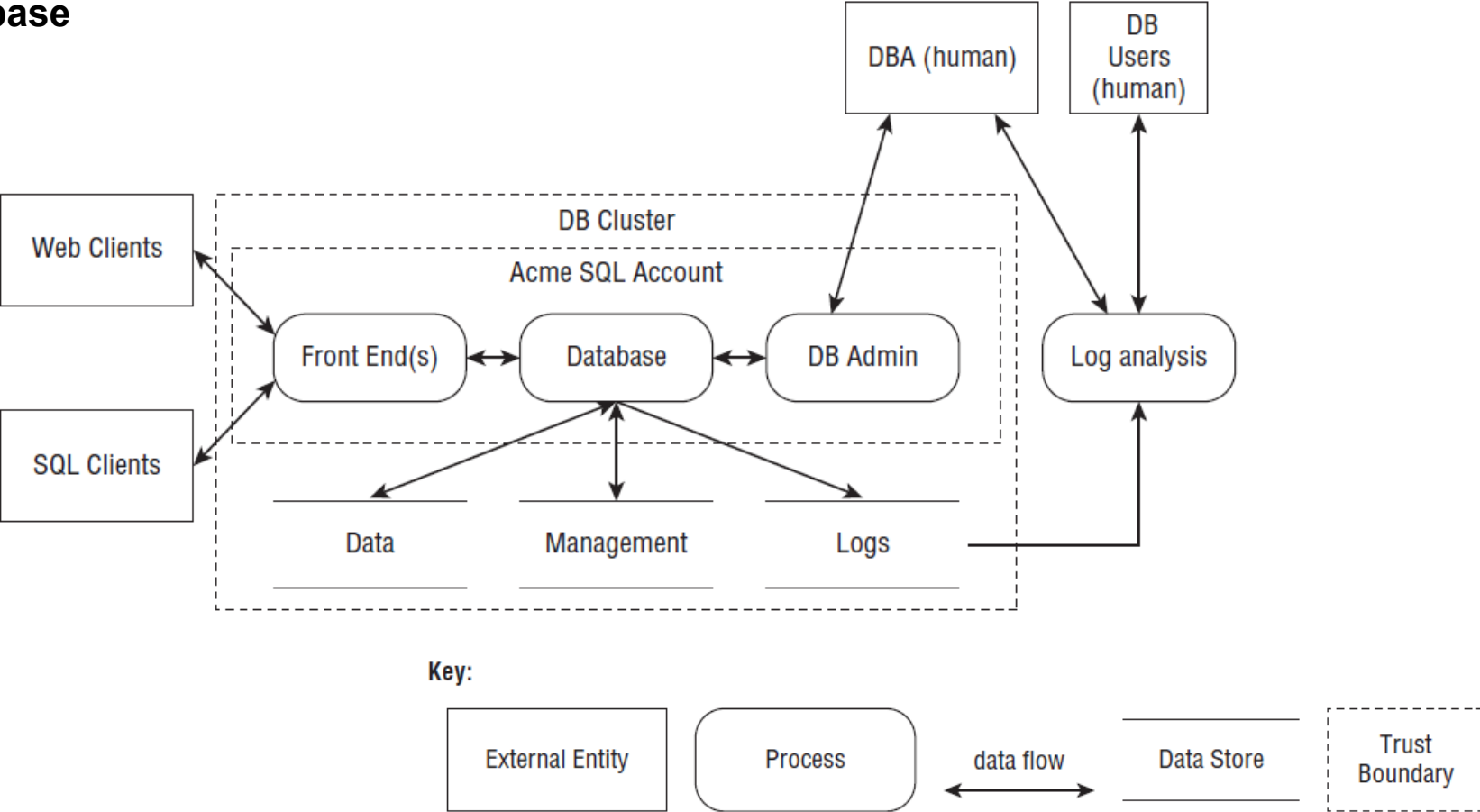
Abstract programs into:

- **Processes:** your code
- **Data stores:** files, databases, shared memory
- **Data flows:** connect processes to other elements
- **External entities:** everything but your code and data, including people and cloud software

Data Flow Diagram (Example 1)

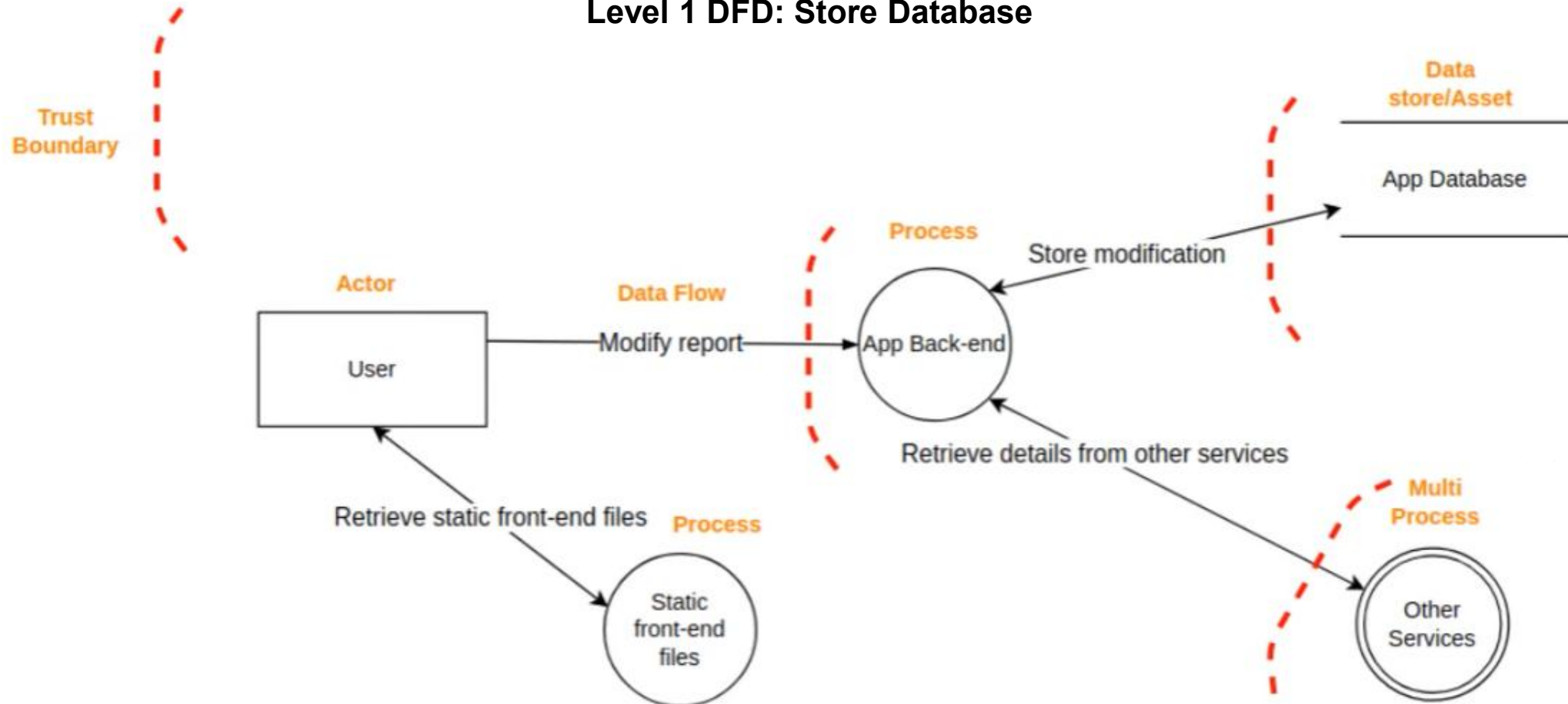
Acme Database

#sdudk



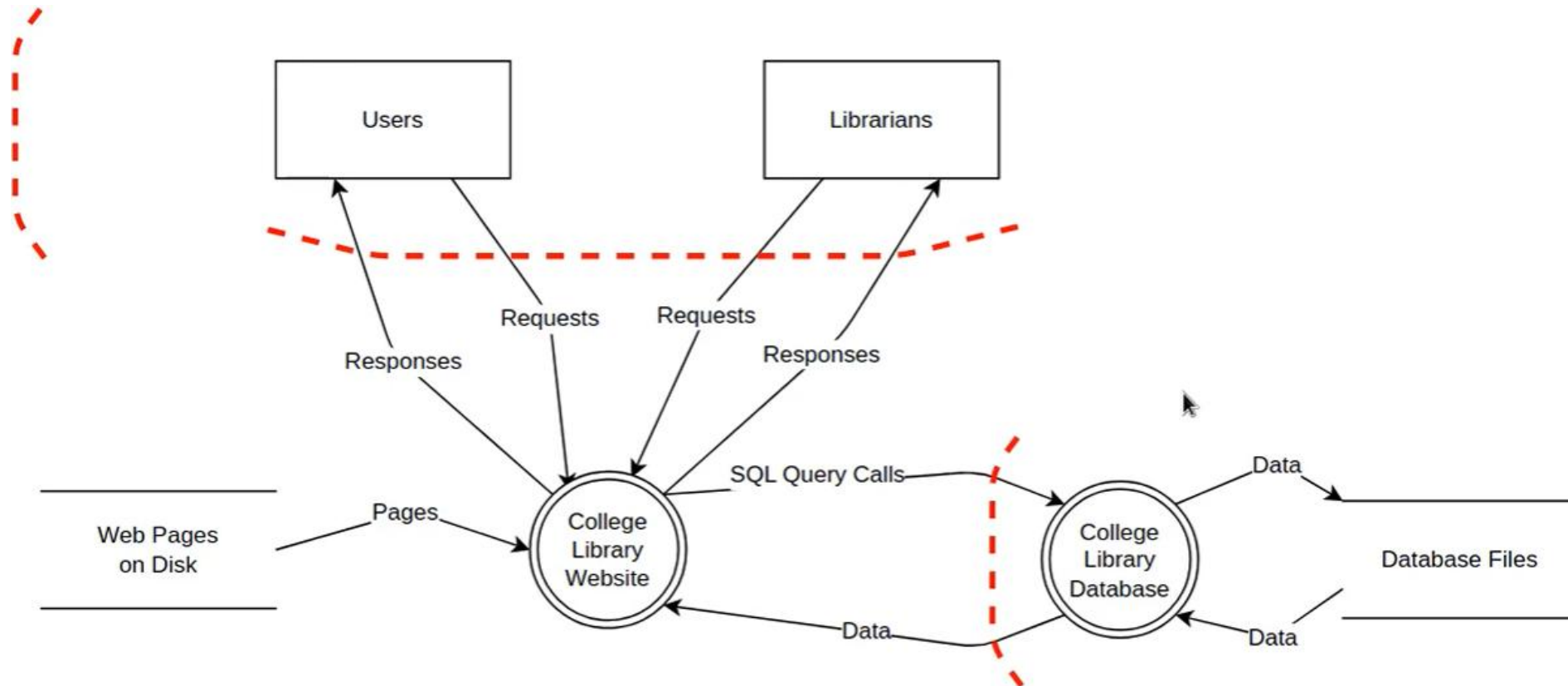
Data Flow Diagram (Example 2)

Level 1 DFD: Store Database



Data Flow Diagram (Example 3)

Level 2 DFD: College Library Website



Trust Boundaries

Sometimes left implicit in development

- Effective threat modeling requires making boundaries explicit

A trust boundary is where two (or more) principals interacts

- i.e., where entities with different privileges interact
- Apps on mobile platforms (two or more)

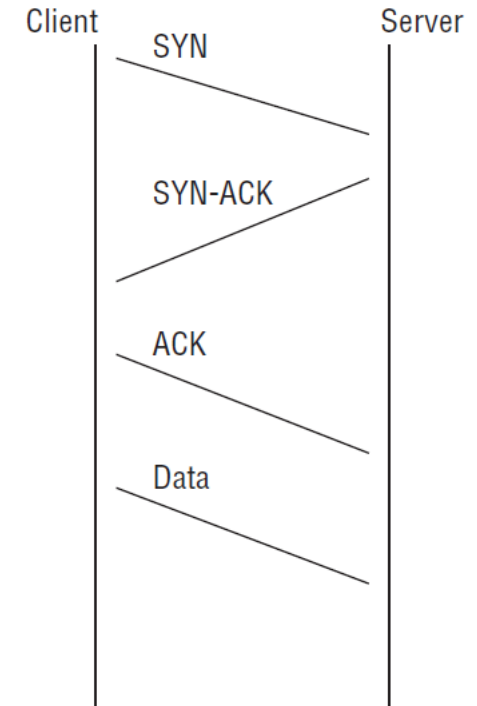
Need to be enforced in some way

- Best to rely on the OS
- Sometimes not possible (e.g., building a database)

Threats will tend to cluster around trust boundaries

Swim Lane Diagrams

- Show two or more entities communicating, each “in a lane”
- Useful for network communication
- Lanes have implicit boundaries between them
- FYI – the swim lane diagram shows an example of 3-way handshake in TCP

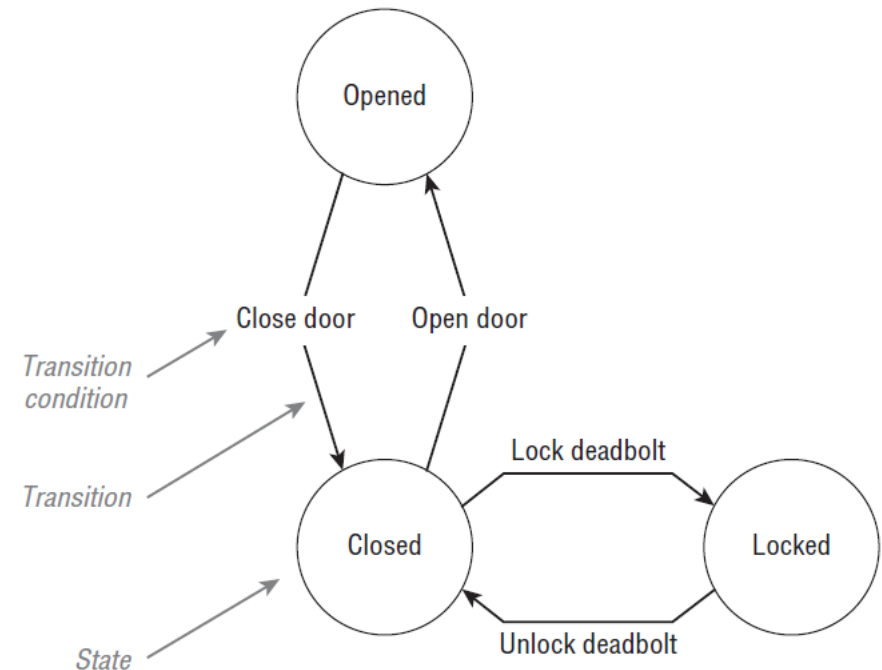


State Machines

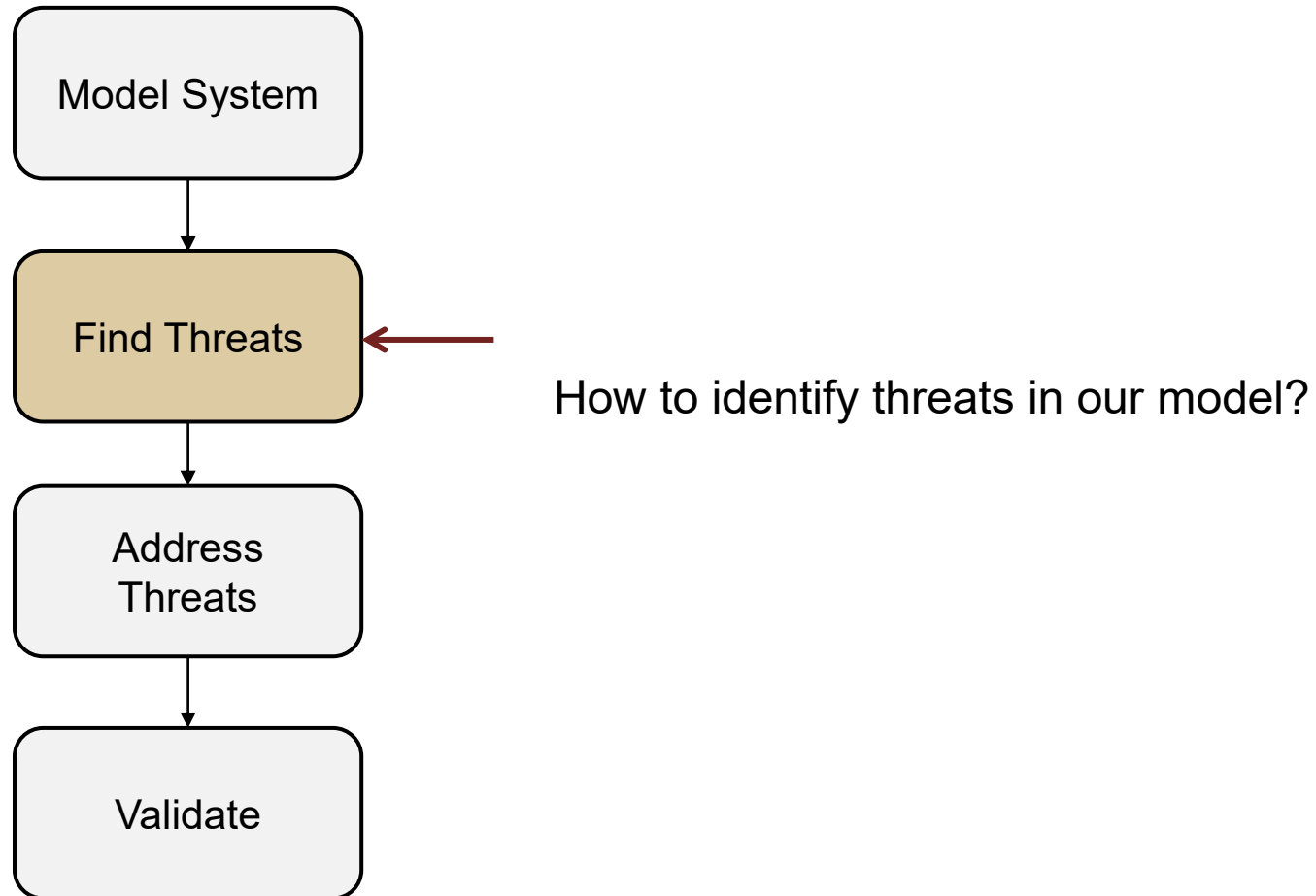
- Helpful for considering what changes security state
 - For example, unauthenticated to authenticated
 - User to root/admin

Other UML diagrams:

- Any UML diagrams can be adapted
 - UML stands for Unified Modeling Language
 - They can be quite complex
- Example use activity diagrams to show process flow
- Data flow diagrams only represents exchange of data



Threat Modeling Flow



Tools to Guide the Threat Search

Tools creating *Structure* as a means to achieve *completeness* and *predictability*

- **Attack libraries** – existing threats/categories as inspiration
- **STRIDE, PASTA, etc** – a general search structure/technique
- **Attack trees** – connecting process and possible threats

Attack libraries

- Collections of threats and vulnerabilities
- Structured and up-to-date information
- Libraries of threats can all help **structure thinking** towards *completeness* and *predictability*.

Examples:

- **OWASP:** [The Open Web Application Security Project](#)
 - Category: [Top ten risks](#)
- **CAPEC:** [Common Attack Pattern Enumeration and Classification](#)
- **CVE:** [Common Vulnerabilities and Exposures](#)
- **CMU:** [Vulnerability Notes Database](#)
- **Exploit-db:** <https://www.exploit-db.com/>
 - The vulnerability description: <https://nvd.nist.gov/vuln>

STRIDE and Attack Trees

STRIDE

STRIDE: A methodology for identifying and categorizing threats

- **S**poofing
 - **T**ampering
 - **R**epudiation
 - **I**nformation Disclosure
 - **D**enial of Service
 - **E**levation of privileges
-
- Business oriented: easier for non-technical persons to relate to
 - Expand (can replace) “may by mechanisms and subsystems” approach
 - Can be used to also identify threats, e.g., as pentest checklist.

STRIDE

Threat	Property Violated	Definition
Spoofing	Authentication	Impersonating something or someone else.
Tampering	Integrity	Modifying data or code
Repudiation	Non-repudiation	Claiming to have not performed an action.
Information Disclosure	Confidentiality	Exposing information to someone not authorized to see it
Denial of Service	Availability	Deny or degrade service to users
Elevation of Privilege	Authorization	Gain capabilities without proper authorization

STRIDE

Spoofing over a network

Threat Example	What the Attacker Does	Notes/Examples
Spoofing a machine	ARP spoofing	
	IP spoofing	
	DNS spoofing	
	DNS compromise	Can be at the TLD, registrar or DNS server
	IP redirection	
Spoofing a person	Take over account	“Stranded in London”
	Set the display name	
Spoofing a role	Declares themselves to be that role	Sometimes opening a special account, setting up a domain/website, other “verifiers”

STRIDE

Tampering with a file

Threat Example	What the Attacker Does	Notes/Examples
Modifying a file...	... which you own and you rely on	
	... which they own and you rely on	
Modifying a file on a server...	...you own	
	...they own (or take over)	
Modifies links or redirects		Redirects are super-common on the web, and often rot away

STRIDE

Repudiation attacks on logs

Threat Example	What the Attacker Does	Notes/Examples
	Discovers there are no logs	
Modifies data flowing over the network	Puts data in the logs to confuse you	</tr></html>

STRIDE

Information disclosure (process)

Threat Example	What the Attacker Does	Notes/Examples
Extracts user data	Exploits bugs like SQL injection to read db tables	Can find this by looking to data stores, but here the issue is the process returning data it shouldn't
	Reads error messages	
Extracts machine secrets	Reads error messages	Cannot connect to database 'foo' as user 'sql' with password '&IO*(^&'
	Exploits bugs	"Heartbleed"

STRIDE

Information disclosure (data flow)

Sub-category	What the Attacker Does
Network	Read data on a network
	Redirects traffics to enable reading data on the network
Metadata	Learns secrets by analyzing traffic
	Learns who talks to whom by watching the DNS
	Learns who talks to whom by analyzing social network information

STRIDE

Denial of service

Threat Example	What the Attacker Does	Notes/Examples
Against a process	Absorb memory (ram or disk)	
	Absorb CPU	
	Uses a process as an amplifier	
	Against business logic	“Too many login attempts”
Against a data store	Fills the data store	
	Makes enough requests to slow the system	
Against a data flow	Consumes network resources	

Can be temporary (as attack continues; fill the network) or persist beyond that (fill a disk)

STRIDE

Elevation of privilege (EoP)

Threat Example	What the Attacker Does	Notes/Examples
EoP Against process via corruption	Sends inputs the code doesn't handle properly	Very common, usually high impact
	Gains read/write access to memory	Writing memory more obviously bad
EoP via misused authorization checks		
EoP via buggy authorization checks		Centralizing checking makes consistency, correctness easier
EoP via data tampering	Modify bits on disk	

STRIDE

Using STRIDE

- Consider how each **STRIDE threat** could impact each part of the model

How could a clever attacker spoof certain part of the system?

Could the attacker tamper with it? ... etc.

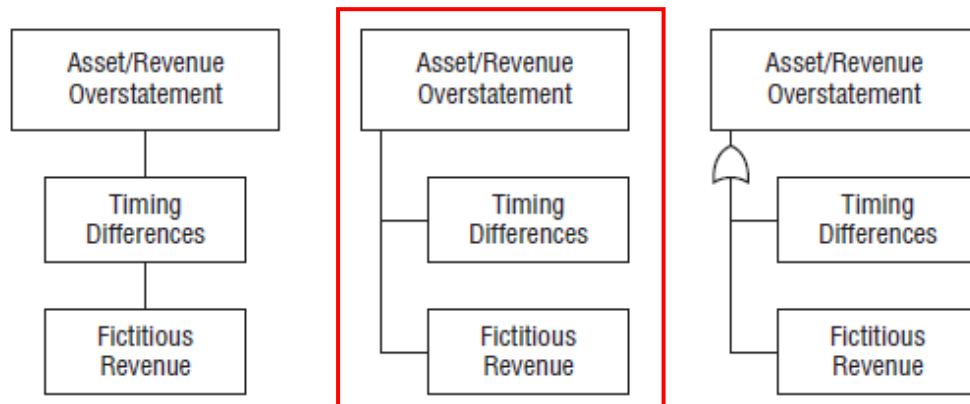
- **Use aids**
 - All available information at your disposal
 - Attack trees
 - Experience, and more of it..

Attack Trees

- A branching, hierarchical data structure that represents a set of potential techniques for exploiting security vulnerabilities.
- The security incident that is the **goal of the attack is represented as the root node** of the tree, and the **ways that an attacker could reach that goal are represented as branches and subnodes** of the tree.
- The final nodes on the paths outward from the root (leaf nodes), represent different ways to initiate an attack.
- The motivation for the use of attack trees is to effectively exploit the information available on attack patterns.

Attack Trees

- **Structured relationship between attack details**
 - Detail (this is a subcategory of that)
 - Present as outline, bigger picture
 - Creation vs. use



Three representations of a tree

Goal: Access to the building

1. Go through a door
 - a. When it's unlocked:
 - i. Get lucky.
 - ii. Obstruct the latch plate (the "Watergate Classic").
 - iii. Distract the person who locks the door at night.
 - b. Drill the lock.
 - c. Pick the lock.
 - d. Use the key.
 - i. Find a key.
 - ii. Steal a key.
 - iii. Photograph and reproduce the key.
 - iv. Social engineer a key from someone.
 1. Borrow the key.
 2. Convince someone to post a photo of their key ring.
 - e. Social engineer your way in.
 - i. Act like you're authorized and follow someone in.
 - ii. Make friends with an authorized person.
 - iii. Carry a box, a cup of coffee in each hand, etc.
2. Go through a window.
 - a. Break a window.
 - b. Lift the window.
3. Go through a wall.
 - a. Use a sledgehammer or axe.
 - b. Use a truck to go through the wall.

Attack Trees

▪ Example:

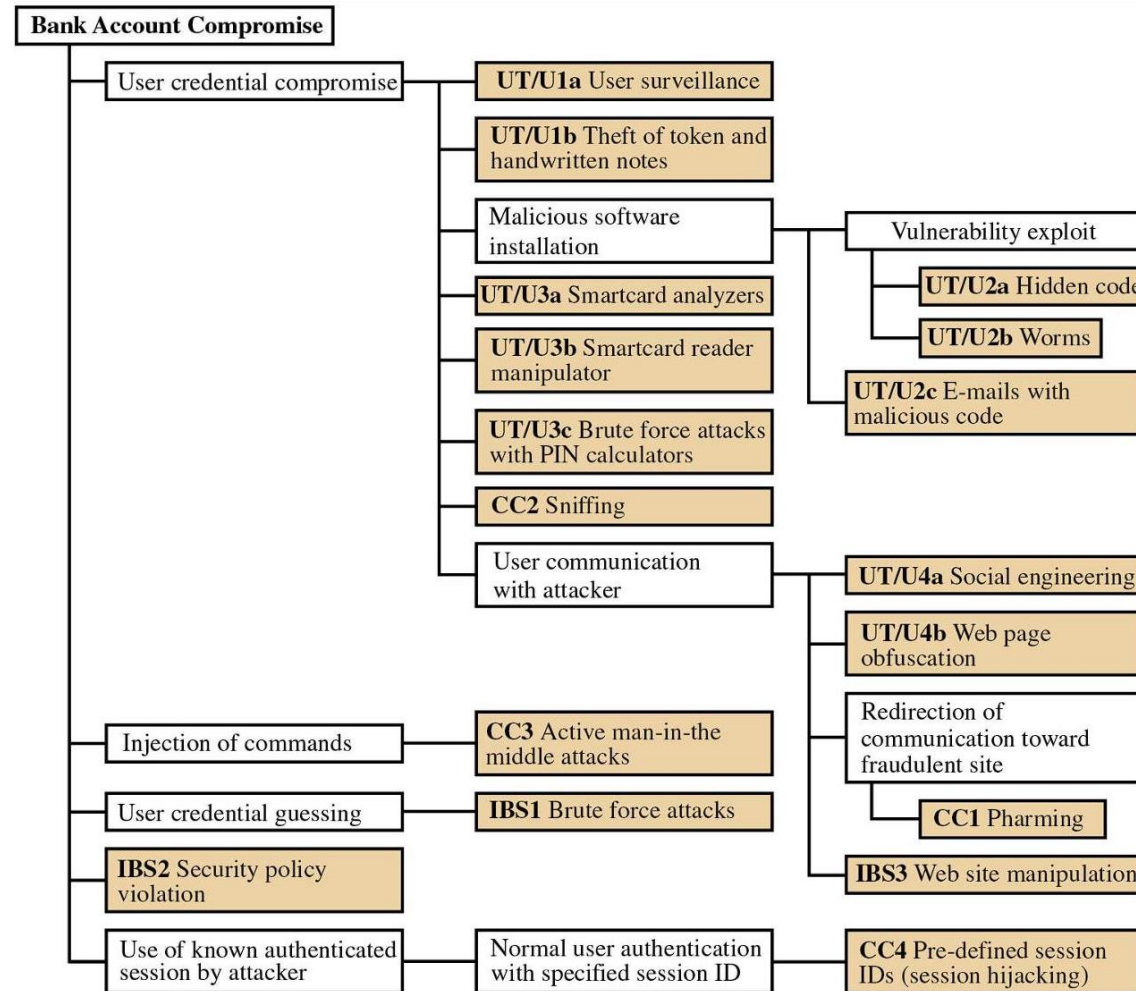


Figure: An Attack Tree for Internet Banking Authentication

Attack Trees

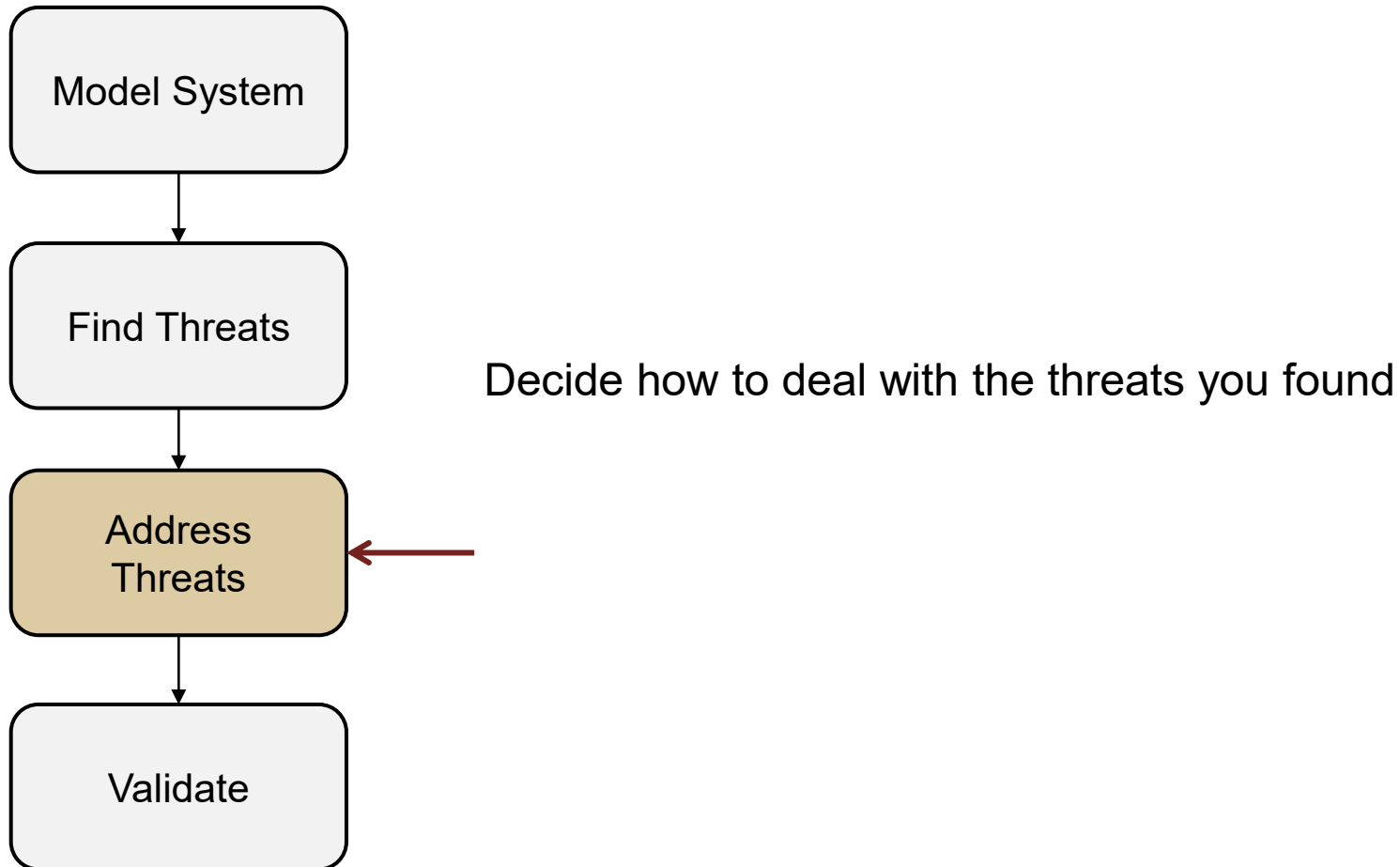
Using an Attack Tree

- Find an appropriate tree
 - Web search
- Iterate through your diagram and tree
 - “Does this apply here”?
 - More precise iteration is more useful when you’re learning, or for high-stakes analysis

Creating attack trees

- For a project or general use
- Steps:
 - Choose a representation, create a root node (goal, “Get root”), add subnodes, consider completeness, prune and check.

Threat Modeling Flow



Threat Mitigation Strategies

For each threat in your list, you need to find a strategy.

- **Mitigating:** make it harder for the attacker to take advantage (default)
- **Eliminating:** remove the functionality responsible
- **Transferring:** let somebody else handle the problem
- **Accepting:** if the threat cannot be addressed in another, reasonable manner

There are almost always ***tradeoffs*** involved, when deciding on how to deal with a threat!

Threat Mitigation Strategies

Addressing Integrity (Spoofing Issues)

- Use permissions as provided
- Cryptography is required over a network, e.g., SSL/TLS for authenticating computers
- Authenticating bits (files, messages, etc), e.g., Digital signatures, Hashes (appropriately managed)
- Implementing a permission system is hard, e.g., Lots of documented mistakes, Password, MFA, etc

Addressing Repudiation

- Logs, e.g., Logging, Log analysis tools, Secure log storage
- Secure time stamps
- Trusted third parties

Threat Mitigation Strategies

Fix It!

- The best way to remove security bug is to remove the functionality
E.g., if SSL dosen't have a "heartbeat" message, the "heartbleed" bug couldn't exist.
- If you don't remove it, you can only mitigate, not fix. More mitigations below:

More Technical Mitigations

Threat	Mitigation Technology	Developer Example	Sysadmin Example
Spoofing	Authentication	Digital signatures, Active directory, LDAP	Passwords, crypto tunnels
Tampering	Integrity, permissions	Digital signatures	ACLs/permissions, crypto tunnels
Repudiation	Fraud prevention, logging, signatures	Customer history risk management	Logging
Information disclosure	Permissions, encryption	Permissions (local), PGP, SSL	Crypto tunnels
Denial of service	Availability	Elastic cloud design	Load balancers, more capacity
Elevation of privilege	Authorization, isolation	Roles, privileges, input validation for purpose, (fuzzing*)	Sandboxes, firewalls

* Fuzzing/fault injection is not a mitigation, but a great testing technique. See chapter 8, Threat Modeling for more.

Threat Prioritization Strategies

- Wait to see
- Easy fixes first
- Threat ranking with bug bar
- Cost/damage estimation approaches
- **DREAD**
 - **D**amage Potential
 - **R**eproducibility
 - **E**xploitability
 - **A**ffected Users
 - **D**iscoverability

For each scale from high to low severity, and assign a point for each score

Threat Prioritization Strategies

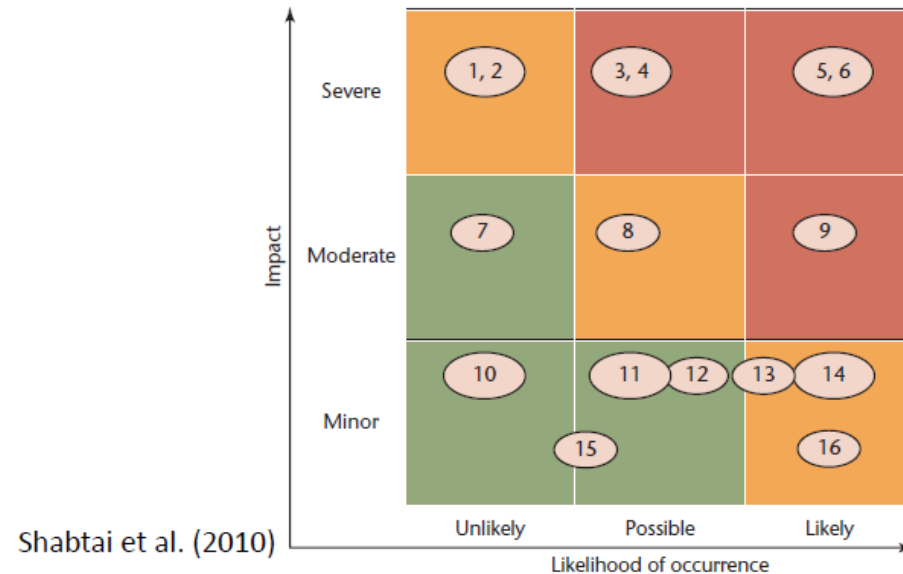
Example: For Threat 1

- Damage
 - 3 points – **High**: An attacker can run as administrator
 - 2 points – **Medium**: Revealing sensitive data
 - 1 points – **Low**: Revealing data
- For this example system, Damage for Threat 1 is given 1 point (Low).
- Sum the score and get average score for the Threat 1
- Similarly, define the scores for each remaining threats “READ” from “DREAD”
- READ: Reproducability, Exploitability, Affected users, Discoverability.

Threat Prioritization Strategies

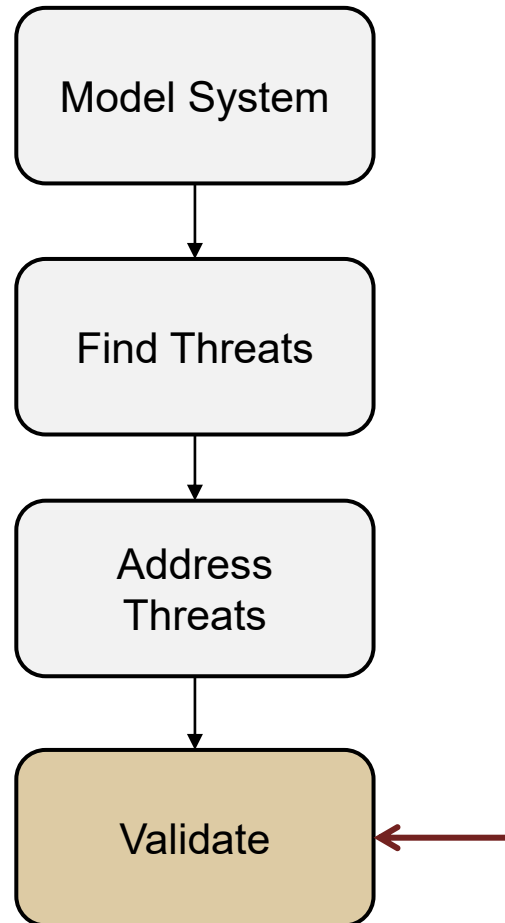
Another approach

- For each threat define a scale for:
 - Impact to system/data: e.g., Minor, moderate, and severe
 - Likelihood of occurrence: e.g., Unlikely, possible and likely
 - Plot it as done below:



In the paper titled "Google android: A comprehensive security assessment"

Threat Modeling Flow – Validation



Quality Assurance for Threat Models

Threat Modeling Flow – Validation

Two Things to Validate:

- The threat model itself
- That the identified threats are addressed – Think “Quality Assurance” for your model.

Validating the Threat Model

- Gather expertise, i.e., everyone involved
- Is the model complete?
- Is the model accurate?
- Does it cover all the security decisions we made?
- Can we go on a newer version of the model?
- When necessary, update!

Threat Modeling Flow – Validation

Continue – Validating the Threat Model: Go through the diagram again, asking

- Have we checked every part for threats?
- Have we considered relevant attack trees?
- Have we gone through the whole STRIDE list?
- Have we made a complete list of threats?
 - i.e., do we have an issue in the bug tracker for each threat?
- Have we listed all assumptions?

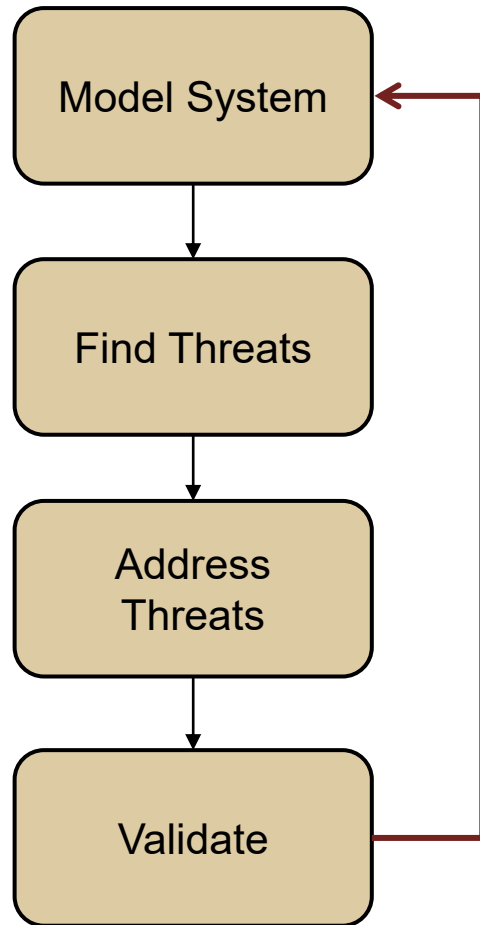
Threat Modeling Flow – Updating the Model

- **Again:** focus on data flow, not control flow
- But keep in mind that every transition indicates processing
- Vague phrasings indicates a need for more detail
- When explaining the diagram, do you add information?
- A discussion could indicate an incomplete diagram
- A data flow model should have no dead ends.

Validating all Threats are Addressed

- Check that the model matches reality
- Do you have documentation for each threat about:
 - Risks, strategy addressing the risk, and test being available for each one?
- Has every threat issue been closed? Are the tests part of the development and/or deployment procedure?

Threat Modeling Flow – Summary



And never forget...

- It is a process...
- You never cover every angle
- Your system changes over time
- Attack patterns change
- New attacks come into focus
- ... the process can continue for the deployed product.

Thank You!